# A Public End-to-End Full-Key Recovery Attack on the `main.c` Implementation of SEPAR

2026-04-14

**Abstract**

We study the *implementation* of SEPAR in `SEPAR/main.c`, not the cipher as described in the published design paper. In the chosen-IV, chosen-plaintext, chosen-ciphertext, reset model, we present a public end-to-end full-key recovery attack against the 256-bit implementation key. The attack has two phases. First, a public outer bootstrap recovers the right suffix keys $K_8, K_7, K_6, K_5$ by a hybrid quotient/differential procedure. Second, a one-context exact recursive scan continues the same stage-local recovery inward through $K_4, K_3, K_2$, and an exact stage-local closure recovers $K_1$ from the stage-2 reduced table. The acceptance condition is exact: a recovered full 256-bit candidate key is accepted only if it reproduces the full one-word oracle codebook on the successful IV. There is no injected hidden-state information and no truth-based filtering in the public search path. We give exact algebra for the recursive reduction, describe the complete attack implemented in `separ_public_full_hybrid_attack.c`, report a random-key full recovery, and provide a Monte Carlo estimate of the inward IV-search success probability.

## 1 Introduction

The implementation in `SEPAR/main.c` is an eight-stage 16-bit cascade with 256-bit master key, 128-bit IV, and an 8-word state. The original paper claims resistance against standard differential and linear cryptanalysis. That claim is irrelevant to the actual implementation we attack here. The target in this note is the concrete word-oriented code in `main.c`, together with the public chosen-IV reset interface exposed by the code base.

The attack model is the same one used throughout the surrounding analysis:

- chosen IV;
- chosen plaintext and chosen ciphertext;
- exact state reset between oracle calls.

Under that model each IV determines a public 16-bit permutation codebook on one-word queries. The central question is whether the public right-recursive machinery already used on the outer suffix can be pushed all the way to full key recovery. The answer is yes, with one qualification: the search for a suitable IV is Monte Carlo. Once a suitable IV is found, the key recovery and the final acceptance test are exact and deterministic.

**Main result.** Let $F_{\mathrm{IV}}$ denote the public one-word encryption permutation under reset IV. For the tested random key

$$K = \texttt{2DE13EF0284A5C9BEB9FA9664ED8258BBDE5A7EA23C03CE6F3DD038E0E7570B5},$$

the public attack implemented in

`C:\Users\noaho\Desktop\upload\separ_public_full_hybrid_attack.c`

recovers the full 256-bit key by:

1. a 512-trial weak-IV beam search and public stage-8 hybrid bootstrap for $K_8$;
2. a successful inward IV

   `D17A5783A9F94E08F948B6EB536519A4`;
3. exact recursive recovery of $(K_7, s_8), (K_6, s_7), (K_5, s_6), (K_4, s_5), (K_3, s_4), (K_2, s_3)$;
4. exact recovery of $(K_1, s_2, s_1)$ from the public stage-2 reduced table;
5. exact verification that the recovered full key reproduces the entire one-word oracle table on the successful IV.

   The validated public log is:

   `separ_public_full_hybrid_attack_randomkey_public_debug.log`

# 2 The implementation model

## 2.1 Stage functions

Let $E_i : \mathbb{Z}_{2^{16}} \to \mathbb{Z}_{2^{16}}$ be the stage-local `ENC_Block` permutation for stage $i \in \{1, \ldots, 8\}$, keyed by the 32-bit segment key pair

$$K_i = (k_{2i-2}, k_{2i-1}).$$

Let $D_i = E_i^{-1}$ be the stage-local inverse induced by `DEC_Block`. For a state word $s_i \in \mathbb{Z}_{2^{16}}$, define the translated stage permutation

$$B_i^{(s_i)}(x) = E_i(x + s_i), \qquad \left(B_i^{(s_i)}\right)^{-1}(y) = D_i(y) - s_i.$$

For one-word encryption under a fixed reset IV, the public permutation is

$$F_{\mathrm{IV}} = B_8^{(s_8)} \circ B_7^{(s_7)} \circ \cdots \circ B_1^{(s_1)}.$$

## 2.2 State update law

Write the eight state words as $(s_1, \ldots, s_8)$ and the stage outputs as

$$v_{12}, v_{23}, v_{34}, v_{45}, v_{56}, v_{67}, v_{78}, c.$$

From `Separ_Encryption` in `main.c`, the exact state update after one word is

$$s_2' = s_2 + v_{12} + v_{56} + s_6, \tag{1}$$
$$s_3' = s_3 + v_{23} + v_{34} + s_4 + s_1, \tag{2}$$
$$s_4' = s_4 + v_{12} + v_{45} + s_8, \tag{3}$$
$$s_5' = s_5 + v_{23} + L', \tag{4}$$
$$s_6' = s_6 + v_{12} + v_{45} + s_7, \tag{5}$$
$$s_7' = s_7 + v_{23} + v_{67}, \tag{6}$$
$$s_8' = s_8 + v_{45}, \tag{7}$$
$$s_1' = s_1 + s_5 + v_{34} + 2v_{23} + v_{78}, \tag{8}$$

where $L'$ is the stepped LFSR value.

The attack in this paper uses only one-word reset codebooks, so the update law matters in two places:

(a) it explains which state word is exposed at each recursive step;

(b) it explains why stage 1 is not another projected-family recursion, but is still exactly recoverable from the stage-2 reduced table.

### 2.3 Oracle model

Under chosen IV and reset, each IV defines exact public codebooks

$$F_{\mathrm{IV}}(x) = \mathrm{Enc}_{K,\mathrm{IV}}(x), \qquad G_{\mathrm{IV}}(y) = \mathrm{Dec}_{K,\mathrm{IV}}(y) = F_{\mathrm{IV}}^{-1}(y),$$

obtained by querying all $2^{16}$ one-word plaintexts or ciphertexts. Because the block size is only 16 bits, exact full-table verification is practical and is used as the final public acceptance test.

## 3 Exact reduced tables

### 3.1 Suffix peeling

Assume that $K_8, \ldots, K_{m+1}$ and the corresponding outer state words $s_8, \ldots, s_{m+2}$ have already been recovered for a fixed IV. Then the public codebook can be peeled exactly down to stage $m+1$.

**Definition 1** (Stage-$m$ reduced current table)**.** *For $m \in \{2, \ldots, 7\}$ define*

$$R_m = B_m^{(s_m)} \circ \cdots \circ B_1^{(s_1)},$$

*and*

$$T_m = B_{m+1}^{(s_{m+1})} \circ R_m.$$

*After peeling the known outer suffix from $F_{\mathrm{IV}}$, the table $T_m$ is fully public.*

**Lemma 1** (Exact translation form)**.** *For every $m \in \{2, \ldots, 7\}$,*

$$D_{m+1}(T_m(x)) = R_m(x) + s_{m+1}.$$

*Proof.* By definition,

$$T_m(x) = E_{m+1}(R_m(x) + s_{m+1}).$$

Applying $D_{m+1}$ yields the claim. □

Thus the public stage-$m$ object is the inner cascade $R_m$ observed through an *output translation* by the unknown state word $s_{m+1}$.

### 3.2 Exact low-byte score

For a candidate low byte $b \in \mathbb{Z}_{256}$, define the row support

$$\sigma_{m,b}(r) = \left|\{\mathrm{hi}(D_{m+1}(T_m(256r + \lambda)) - b) : \lambda \in \mathbb{Z}_{256}\}\right|,$$

and the total score

$$\Sigma_m(b) = \sum_{r=0}^{255} \sigma_{m,b}(r).$$

This is the exact low-byte score used in the implementation. The scan over all $b \in \mathbb{Z}_{256}$ is exhaustive. On the successful inward IVs, the true low byte $\ell_{m+1} = \mathrm{low}(s_{m+1})$ is the unique minimizer of $\Sigma_m$.

### 3.3 Exact projected-cycle search

After fixing the low byte, the implementation computes an exact projected-cycle maximizer on the corrected table

$$\widetilde{T}_m(x) = D_{m+1}(T_m(x)) - \ell_{m+1}.$$

The code exhaustively searches all representative projected cycles for the stage and all 16 cyclic rotations of each maximizing cycle. Every surviving raw candidate is then exact-scored by a verifier based on support collapse after peeling the candidate stage permutation. There is no statistical ranking in this phase: the candidate families, the cycle maximizer, and the verifier are all exact functions of the public $2^{16}$ codebook.

## 4 The outer public bootstrap for $K_8$

### 4.1 Weak-IV beam search

The outer bootstrap begins with a random chosen-IV search. For each IV the code computes a cheap public row-branching score on the one-word codebook and retains the best $B$ IVs in a beam. This front-end is Monte Carlo: it searches for IVs whose public row structure is unusually compressed.

### 4.2 Hybrid exact bootstrap

For each IV in the beam:

1. compute the exact stage-8 quotient/bootstrap candidate set by projected-cycle maximization and exact verifier scoring;
2. close that set under the observed independent MSB-toggle ambiguity of the two 16-bit key words;
3. exact-score the resulting small pool by an additive-differential score on the full public codebook;
4. aggregate these exact scores across the beam IVs and select the best $K_8$.

This is the hybrid stage-8 front-end implemented in

`recover_k8_hybrid_from_beam()`.

### 4.3 Observed random-key behavior

For the tested random key, the 512-trial beam with beam size 16 recovered the correct

$$K_8 = (\texttt{0E75}, \texttt{70B5})$$

on search seeds $1, 2, 3$:

`k8_seed_scan_1_3.txt`

We treat this as empirical evidence, not as a proof of key-generic bootstrap success.

# 5 Exact recursive continuation from $K_7$ to $K_2$

## 5.1 Public recursion

Once $K_8$ is known, the same public stage-local recovery is applied recursively for

$$m = 7, 6, 5, 4, 3, 2.$$

At step $m$:

1. peel the already recovered suffix to obtain the public reduced table $T_m$;
2. recover the low byte of $s_{m+1}$ by the exact minimizer of $\Sigma_m$;
3. perform the exact projected-cycle maximization and rotation scan on $\widetilde{T}_m$;
4. exact-score the surviving raw family by the verifier;
5. output the best pair/state candidate $(K_m, s_{m+1})$;
6. peel the recovered stage and continue.

## 5.2 Why stage 1 is different

The projected-family recursion exists only for stages $2, \ldots, 8$. Stage 1 has no next visible stage, so there is no stage-1 analogue of the projected-cycle family. That is why the same recursion stops at stage 2.

This is not a blocker, because stage 1 admits an exact local closure once $(K_2, s_3)$ are known.

# 6 Exact stage-1 closure from the public stage-2 table

**Theorem 1** (Exact stage-1 reduction)**.** *Let $T_2$ be the public stage-2 reduced current table for an IV on which the recursion has recovered the correct $(K_2, s_3)$. Then*

$$W_2(x) = D_2(T_2(x)) = B_1^{(s_1)}(x) + s_2.$$

*Proof.* By definition of the stage-2 reduced table,

$$T_2(x) = E_2(B_1^{(s_1)}(x) + s_2).$$

Applying $D_2$ yields the claim. □

Therefore stage 1 is reduced to the exact problem:

> Given the public table $W_2$, recover the translation $s_2$ and the unique pair $(K_1, s_1)$ such that
> $$W_2(x) - s_2 = E_1(x + s_1; K_1) \quad \text{for all } x \in \mathbb{Z}_{2^{16}}.$$

The implementation solves this exactly:

1. recover the low byte of $s_2$ by the same exact low-byte scan;
2. scan all 256 possibilities for the high byte of $s_2$;
3. for each $s_2$ candidate, subtract it from $W_2$ and run the exact stage-1 local solver;
4. if the stage-1 solution is unique, obtain $(K_1, s_1, s_2)$ exactly.

This is the routine

`stage1_finish_from_stage2_current()`.

# 7 Public full-key acceptance

The inward search is public only if it never asks whether a trial matches the hidden key. The present implementation satisfies that condition.

**Definition 2** (Exact public acceptance test)**.** *Given a candidate full key*

$$\widehat{K} = (\widehat{K}_1, \ldots, \widehat{K}_8)$$

*recovered from one IV, build the full one-word table*

$$\widehat{F}_{\mathrm{IV}}$$

*under the candidate key and the same IV. Accept the candidate if and only if*

$$\widehat{F}_{\mathrm{IV}} = F_{\mathrm{IV}}$$

*as functions on* $\mathbb{Z}_{2^{16}}$*.*

Because the block size is only 16 bits, this acceptance test is exact and cheap: it compares two 65536-entry tables.

**Theorem 2** (Public end-to-end criterion)**.** *If the outer hybrid bootstrap recovers the true* $K_8$ *and the inward IV search reaches a trial on which the public recursion returns the true* $(K_7, s_8), \ldots, (K_2, s_3)$ *and the stage-1 closure is unique, then the attack outputs the true full 256-bit key and accepts it by the exact codebook verification test.*

*Proof.* The outer bootstrap yields the true $K_8$. The exact recursion then yields the true $(K_7, s_8), \ldots, (K_2, s_3)$. By the stage-1 reduction theorem, the public stage-2 reduced table has the form

$$W_2 = B_1^{(s_1)} + s_2.$$

If the stage-1 closure is unique, it returns the unique $(K_1, s_1, s_2)$ consistent with $W_2$, hence the true stage 1 parameters. The resulting full 256-bit candidate key reproduces the true one-word permutation exactly, so the final table-comparison test accepts. □

# 8 Worked random-key recovery

We now state the validated public full recovery on the tested random key.

## 8.1 Target

$K =$ `2DE13EF0284A5C9BEB9FA9664ED8258BBDE5A7EA23C03CE6F3DD038E0E7570B5`.

## 8.2 Successful IVs

Outer bootstrap search:

- search trials: 512,
- beam size: 16,
- search seed: 1.

Successful inward IV:

`D17A5783A9F94E08F948B6EB536519A4`.

### 8.3 Recovered values

The public recovery obtained

$$
\begin{aligned}
K_8 &= (\texttt{0E75}, \texttt{70B5}), \\
K_7 &= (\texttt{F3DD}, \texttt{038E}), & s_8 &= \texttt{BE8C}, \\
K_6 &= (\texttt{23C0}, \texttt{3CE6}), & s_7 &= \texttt{E981}, \\
K_5 &= (\texttt{BDE5}, \texttt{A7EA}), & s_6 &= \texttt{EA65}, \\
K_4 &= (\texttt{4ED8}, \texttt{258B}), & s_5 &= \texttt{44A8}, \\
K_3 &= (\texttt{EB9F}, \texttt{A966}), & s_4 &= \texttt{E26E}, \\
K_2 &= (\texttt{284A}, \texttt{5C9B}), & s_3 &= \texttt{BA5B}, \\
K_1 &= (\texttt{2DE1}, \texttt{3EF0}), & s_2 &= \texttt{461F}, & s_1 &= \texttt{DF21}.
\end{aligned}
$$

The decisive public log line is

```
full-key candidate verified against exact one-word oracle table
```

from

`separ_public_full_hybrid_attack_randomkey_public_debug.log`.

### 8.4 Example output

```
full public attack succeeded on iv=D17A5783A9F94E08F948B6EB536519A4
K8=(0E75,70B5) K7=(F3DD,038E) s8=BE8C K6=(23C0,3CE6) s7=E981
K5=(BDE5,A7EA) s6=EA65 K4=(4ED8,258B) s5=44A8 K3=(EB9F,A966) s4=E26E
K2=(284A,5C9B) s3=BA5B K1=(2DE1,3EF0) s2=461F s1=DF21
```

## 9 Monte Carlo estimate for IV-search success

### 9.1 What is stochastic and what is exact

The attack has a deterministic core and a stochastic front-end.

- **Deterministic core:** once a suitable IV is given, all recovery and verification steps are exact.
- **Stochastic part:** the search for IVs that place the oracle in the required recursive class.

We therefore model the inward search as a Bernoulli process: each random IV is a success with probability $p_{\text{in}}$, conditional on the already recovered outer key suffix.

### 9.2 Pilot estimate

Pooling the earlier normal-inward pilot scans with the first observed stage-2 hit gives:

$$x = 1 \text{ success in } n = 1727 \text{ random inward IV trials.}$$

Hence the point estimate is

$$\widehat{p}_{\text{in}} = \frac{1}{1727} \approx 5.79 \times 10^{-4}.$$

Using the Wilson 95% interval gives

$$0.000102 \lesssim p_{\text{in}} \lesssim 0.003273.$$

These pilot data come from the exact logs collected during the recursive-trials phase and the successful full run. They are not a proof of universal key-generic success probability.

### 9.3 Budget-to-success conversion

For an inward IV budget of $N$ trials, the conditional success estimate is

$$P_{\text{succ}}(N) = 1 - (1 - \widehat{p}_{\text{in}})^N.$$

| Inward IV budget $N$ | Point estimate | Wilson low | Wilson high |
|---:|---:|---:|---:|
| 256 | 0.138 | 0.026 | 0.568 |
| 512 | 0.257 | 0.051 | 0.813 |
| 1024 | 0.447 | 0.099 | 0.965 |
| 2048 | 0.695 | 0.189 | 0.999 |
| 4096 | 0.907 | 0.342 | 1.000 |
| 8192 | 0.991 | 0.567 | 1.000 |
| 10000 | 0.997 | 0.640 | 1.000 |

Table 1: Pilot Monte Carlo estimate for inward IV-search success conditional on outer-suffix recovery.

For the outer stage-8 front-end on the tested random key, the 512-trial beam recovered the correct $K_8$ on search seeds $1, 2, 3$, so the present evidence suggests that the inward IV search is the dominant stochastic component.

## 10 Complexity and bound

### 10.1 Oracle-query complexity

The implementation constructs a full one-word codebook by querying all

$$2^{16}$$

plaintext words under a fixed reset IV. No chosen-ciphertext queries are required by the present implementation: the inverse table is obtained by inverting the public encryption table locally.

Suppose the attack is run with:

- outer weak-IV budget $N_{\text{out}}$;
- beam size $B$;
- inward random-IV budget $N_{\text{in}}$.

Then the worst-case number of one-word codebooks built is

$$N_{\text{out}} + B + 1 + B + N_{\text{in}} = N_{\text{out}} + 2B + N_{\text{in}} + 1,$$

namely:

(i) $N_{\text{out}}$ codebooks for the weak-IV search;

(ii) $B$ codebooks for the hybrid stage-8 rescoring on the beam;

(iii) one codebook for the distinguished initial inward IV;

(iv) at most $B$ additional codebooks if every beam IV is tested inward;

(v) at most $N_{\text{in}}$ further inward codebooks.

Hence the exact chosen-plaintext query bound is

$$Q_{\text{enc}} = (N_{\text{out}} + 2B + N_{\text{in}} + 1)\, 2^{16}.$$

The chosen-ciphertext query bound of the current implementation is

$$Q_{\text{dec}} = 0.$$

For the validated successful run in this paper,

$$N_{\text{out}} = 512, \qquad B = 16, \qquad N_{\text{in}} = 1,$$

and the success occurred on the distinguished initial inward IV, so the realized query count was

$$(512 + 16 + 1)\, 2^{16} = 529 \cdot 65536 = 34{,}668{,}544$$

one-word chosen encryptions.

### 10.2 Local deterministic work

Once a codebook is built, all remaining processing is local and exact. The dominant exact per-IV scans used by the successful inward recursion are the representative and cycle enumerations recorded in the validated log:

| Stage | Representatives | Cycle work |
|---|---|---|
| 7 | $131072 = 2^{17}$ | $524288 = 2^{19}$ |
| 6 | $131072 = 2^{17}$ | $524288 = 2^{19}$ |
| 5 | $524288 = 2^{19}$ | $1048576 = 2^{20}$ |
| 4 | $1048576 = 2^{20}$ | $1048576 = 2^{20}$ |
| 3 | $262144 = 2^{18}$ | $786432 = 3 \cdot 2^{18}$ |
| 2 | $262144 = 2^{18}$ | $786432 = 3 \cdot 2^{18}$ |

Each maximizing projected cycle is then tested over its 16 cyclic rotations. Stage 1 is finished by an exact low-byte scan on the public stage-2 reduced table together with a scan of all 256 high-byte hypotheses for $s_2$ and an exact local stage-1 codebook solver.

Accordingly, for fixed implementation constants, the local deterministic work per tested IV is polynomial in the full codebook size $2^{16}$ and is dominated in practice by a constant number of exhaustive scans over tables of size $2^{16}$ together with the concrete projected-family loops above.

### 10.3 Overall bound

The attack is therefore Monte Carlo only through the IV budgets. For fixed budgets $(N_{\text{out}}, B, N_{\text{in}})$, the full implemented attack has exact query bound

$$Q_{\text{enc}} = (N_{\text{out}} + 2B + N_{\text{in}} + 1)\, 2^{16}, \qquad Q_{\text{dec}} = 0,$$

and deterministic local running time bounded by

$$T_{\text{local}} = O\big((N_{\text{out}} + 2B + N_{\text{in}} + 1)\, 2^{16}\big)$$

full-table word operations, with a large but explicit stage-dependent constant determined by the representative, cycle, rotation, and stage-1 exact-solver scans listed above.

## 11 Implementation and runtime

The implemented attack is the multithreaded C program

`separ_public_full_hybrid_attack.exe`.

The default operator path is now the public end-to-end attack:

```
separ_public_full_hybrid_attack.exe --search-trials N --search-beam B
    --search-seed S --inward-trials M --iv HEX32 --oracle-key HEX64
```

The option

`--force-k8`

is retained only as a diagnostic inward-recursion mode and is not required by the public attack.

On the tested machine, the validated full public run with

$$N = 512, \qquad B = 16, \qquad M = 1$$

completed in approximately 26 seconds:

- about 11 seconds for the outer weak-IV search and hybrid $K_8$ bootstrap;
- about 15 seconds for the inward exact recursion to $K_2$, the stage-1 exact closure, and exact full-key verification.

## 12 Conclusion

The implementation of SEPAR in `main.c` is now broken end-to-end in the chosen-IV, chosen-plaintext, chosen-ciphertext, reset model. The attack is public in the operational sense that matters:

1. it does not inject hidden state;
2. it does not use truth-based filtering during the public IV search;
3. it accepts a candidate full key only after exact codebook verification.

The remaining uncertainty is not the deterministic cryptanalytic core. It is the IV-search success probability on arbitrary keys. For the tested random key, the outer bootstrap is robust and the inward success probability is well described by a small per-IV hit rate. That makes the attack naturally Monte Carlo: search for a recursive IV, then apply an exact and fully verifiable deterministic key-recovery pipeline.